

Module 3

STRUCTURAL TESTING

Statement Testing

- Statement testing focuses on covering each line or statement at least once.
- The statement coverage criterion requires each statement to be executed at least once, reflecting the idea that a fault in a statement cannot be revealed without executing the faulty statement.
- **Statement Adequacy Criterion:** Let T be a test suite for a program P . T satisfies the statement adequacy criterion for P , iff, for each statement S of P , there exists at least one test case in T that causes the execution of S .

- Statement Coverage: The statement coverage $C_{\text{statement}}$ of T for P is the fraction of statements of program P executed by at least one test case in T .

$$C_{\text{Statement}} = \frac{\text{number of executed statements}}{\text{number of statements}}$$

- T satisfies the statement adequacy criterion if $C_{\text{Statement}} = 1$.

Branch Testing

- Branch testing is a **white-box testing** technique where the tester designs test cases to ensure that every possible branch (or decision point) in the program's control flow is executed at least once.
- In a program, there are decision points like if statements, switch cases, loops, etc.
- Each decision can lead to multiple branches (e.g., the if condition is true or false).
- Branch testing aims to cover all these branches so that no logical path is left untested.

Condition Testing

- A decision can be composed of a simple condition such as $x < 0$, or of a more complex condition, such as $((x < 0 \text{ AND } y < 0) \text{ OR } (p \geq q))$.
- AND, OR, XOR are the logical operators that connect two or more simple conditions to form a compound condition.
- Focuses on evaluating each condition in a decision statement independently.
- A simple condition is considered covered if it evaluates to true and false in one or more executions of the program in which it occurs.
- A compound condition is considered covered if each simple condition it is comprised of is also covered.

Path Testing

- A path is a sequence of statements and branches from the start to the end of the program (or a module).
- Path testing is a white-box testing technique to design test cases so that every possible execution path through the program is tested at least once.
- **Program Graph:**
 - Given a program written in any programming language, the program graph is a directed graph in which nodes are statement fragments and edges represent flow of control.
 - If i and j are nodes in the program graph, an edge exists from node i to node j iff the statement fragment corresponding to node j can be executed immediately after the statement fragment corresponding to node i .

Ex-Program graph for Triangle program

1 Program triangle2 ` Structured programming version of simpler specification
、

2 Dim a,b,c As Integer

3 Dim IsATriangle As Boolean
、

` Step1: Get Input

4 Output("Enter the 3 integers which are sides of a triangle ")

5 Input(a,b,c)

6 Output("Side A is ",a)

7 Output("Side B is",b)

8 Output("Side C is" c)
、

` Step2:Is A Triangle?

9 If(a<b+c) AND (b<a+c) AND (c<a+b)

10 Then IsATriangle=True

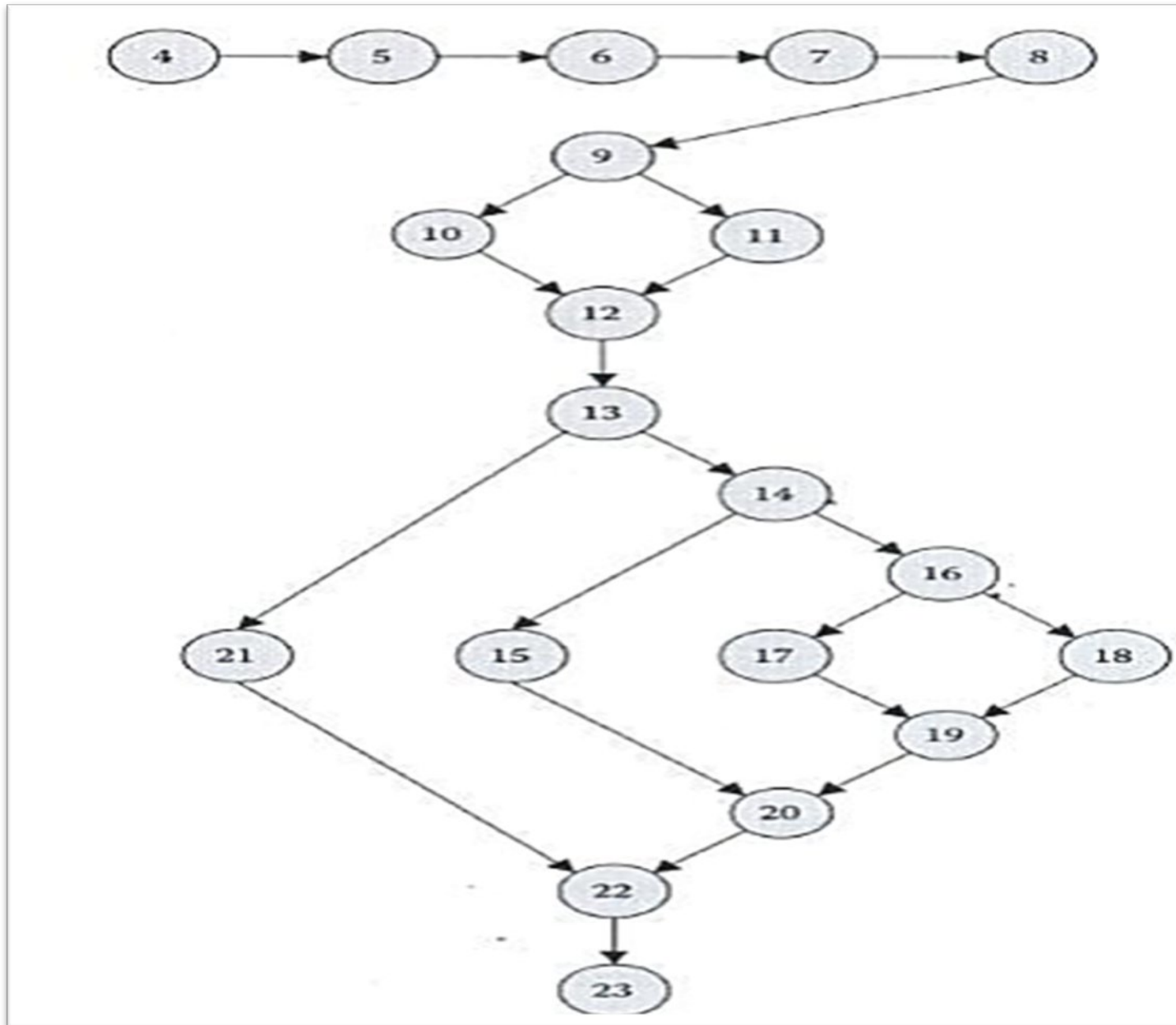
11 Else IsATriangle=False

12 EndIf
、

```
` Step3:Determine Triangle Type
13 If IsATriangle
14 Then If(a=b) AND (b=c)
15 Then Output ("Equilateral")
16 Else If( a!=b) AND (a!=c) AND (b!=c)
17 Then Output ("Scalene")
18 Else Output (" Isoceles")
19 EndIf
20 EndIf
21 Else Output("Not a Triangle")
22 EndIf
`
23 Endtriangle2
```

Here, line numbers refer to statements and statement fragments.

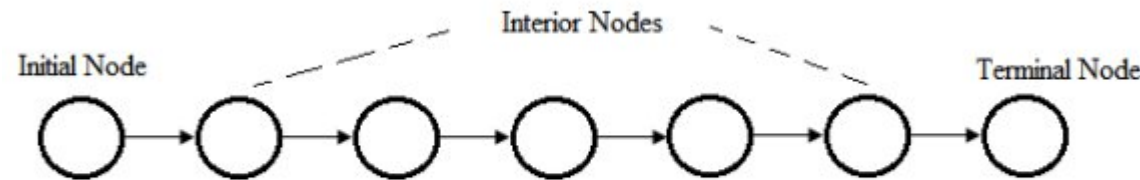
In the program graph, non-executable statements such as variable and type declarations are not included.



Nodes 4 through 8 are a sequence, nodes 9 through 12 are an if-then-else construct, and nodes 13 through 22 are nested if-then-else constructs. Nodes 4 and 23 are the program source and sink nodes, corresponding to the single-entry, single-exit criteria. No loops exist, so this is a Directed acyclic graph.

DD-Path

- Structural testing is based on a construct known as a decision-to-decision path (DDPath). The name refers to a sequence of statements that begins with the "outway" of a decision statement and ends with the "inway" of the next decision statement.
- **Definition: DD-Paths in terms of paths of nodes in a directed graph (Chains)**
- A chain is a path in which the initial and terminal nodes are distinct, and every interior node has indegree = 1 and outdegree = 1



The length (Number of edges) of chain is 6 of above figure

Definition

- A DD-Path is a sequence of nodes in a program graph such that:
- Case 1: It consists of a single node with $\text{indeg}=0$
- Case 2: It consists of a single node with $\text{outdeg}=0$
- Case 3: It consists of a single node with $\text{indeg}\geq 2$ or $\text{outdeg}\geq 2$
- Case 4: It consists of a single node with $\text{indeg}=1$ and $\text{outdeg}=1$
- Case 5: It is a maximal chain of $\text{length}\geq 1$

- Case 1 and Case2 establish a initial and final DD paths of a program graph
- Case 3 deals with a complex nodes ,it ensures that no node is contained in more than one DD path
- Case 4 is needed for short branches, it also preserves the one fragment ,one DD-path principle.
- Case 5 is the normal case ,in which DD path is a single entry,single exist sequence of nodes.

Types of DD-Paths
for fig 1

•Program Graph Nodes	•DD path Name	•Case of Definition
•4	•First	•1
•5-8	•A	•5
•9	•B	•3
•10	•C	•4
•11	•D	•4
•12	•E	•3
•13	•F	•3
•14	•H	•3
•15	•I	•4
•16	•J	•3
•17	•K	•4
•18	•L	•4
•19	•M	•3
•20	•N	•3
•21	•G	•4
•22	•O	•3

DD-Path graph

- Given a program written in an imperative language, the DD-Path graph is the directed graph in which nodes are DD-Paths of its program graph, and edges represent control flow between successor DD-Paths.

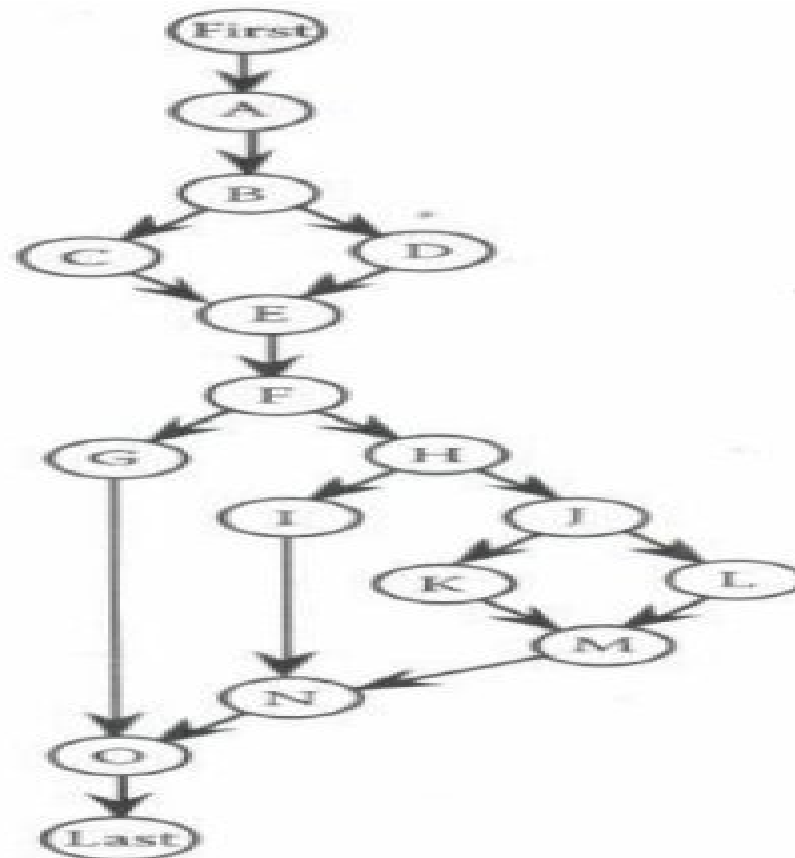


Fig.2: DD-Path graph for the Triangle Program

TEST COVERAGE METRICS or r E.F. Miller Test Coverage Metrics

- Test Coverage Metrics are a device to measure the extent to which a set of test cases covers a program.

Structural Test Coverage Metrics

Metric	Description of Coverage
C_0	Every Statement
C_1	Every DD-Path
C_{1P}	Every predicate to each outcome
C_2	C_1 Coverage + loop coverage
C_d	C_1 Coverage + every dependent pair of DD-Paths
C_{MCC}	Multiple condition coverage
C_{ik}	Every program path that contains up to k repetitions of a loop (usually $k=2$)
C_{stat}	“Statistically significant” fraction of paths
C_∞	All possible execution paths

Metric –Based Testing

- Test coverage metric in table tells us what to test but not how to test it.
- Miller's test coverage metrics are based on program graphs in which nodes are full statements, whereas our formulation allows statement fragments to be nodes.

Statement and Predicate(branch) Testing

- Because our formulation allows statement fragments to be individual nodes , the statement and predicate levels collapse into one consideration.
- If we consider nodes to correspond to full statements, we could execute just one of the decision alternatives and satisfy statement coverage criterion.
- Example: In triangle problem nodes 9, 10, 11, and 12 are a complete if-then-else statement. If nodes correspond to full statements then execute just one of the decision alternatives and satisfy the statement coverage criterion.
- If statement fragments are considered, then divide such a statement into three nodes. Doing so results in predicate outcome coverage.

Statement and Predicate(branch) Testing

- If Interpreted as a Whole Statement (Statement Coverage):
 - You can cover this by executing only one path:
 - Either $9 \rightarrow 10 \rightarrow 12$
 - Or $9 \rightarrow 11 \rightarrow 12$
- If Interpreted as Fragments (Predicate Outcome Coverage):
 - You must execute both paths:
 - $9 \rightarrow 10 \rightarrow 12$ (predicate at node 9 is true)
 - $9 \rightarrow 11 \rightarrow 12$ (predicate at node 9 is false)

DD-Path Testing

- When every DD-Path is traversed(C1 metric) we know that each predicate outcome has been executed, this shows traversing every edge in the DD-Path graph.
- Example: For if-then and if-then-else statements, both the true and the false branches are covered ($C1_p$ coverage). For CASE statements, each clause is covered.
- Longer DD-path generally represent complex computations. For such DD-Path it may be appropriate to apply number of functional tests.

Dependent Pairs of DD-Path

- The dependency among pairs of DD-Paths is the define/reference relationship, in which a variable is defined in one DD-Path and is referenced in another DD-Path.
- Example: In Figure 2, C and H is dependent pair, as are DD-Paths D and H

DD-Path

C

D

H

Action

Defines IsATriangle = TRUE

Defines IsATriangle = FALSE

References IsATriangle

Multiple Condition Coverage

- Multiple Condition Coverage requires every possible combination of conditions in a decision to be tested at least once.
- One possibility is to make truth table.
- In complex decisions with multiple conditions combined using AND (&&), OR (||) MCC ensures that all combinations of condition truth values are tested.
- Example: Compound condition of 2 simple conditions would have 4 rows, yielding 4 test cases, 3 simple conditions would have 8 rows, yielding 8 test cases.

Loop Coverage

- Loop Coverage focuses on ensuring that loops in your program are executed under different conditions.
- Loops occur as : concatenated, nested and knotted.
- Concatenated loops are simply a sequence of disjoint loops.
- Nested loops are such that one is contained inside another.
- Knotted loops :when it is possible to branch into(or out from) the middle of a loop, and these branches are internal to other loops.

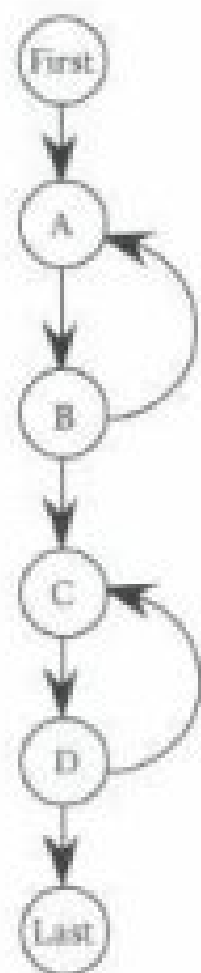
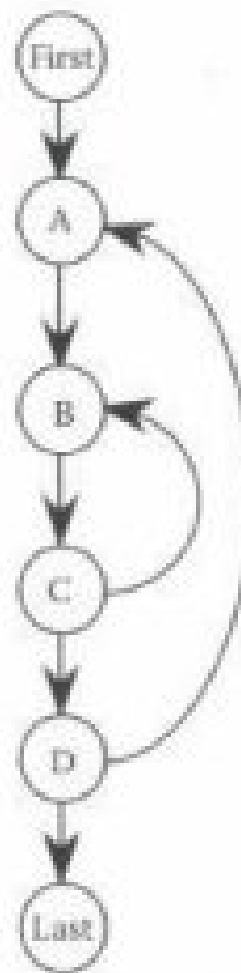
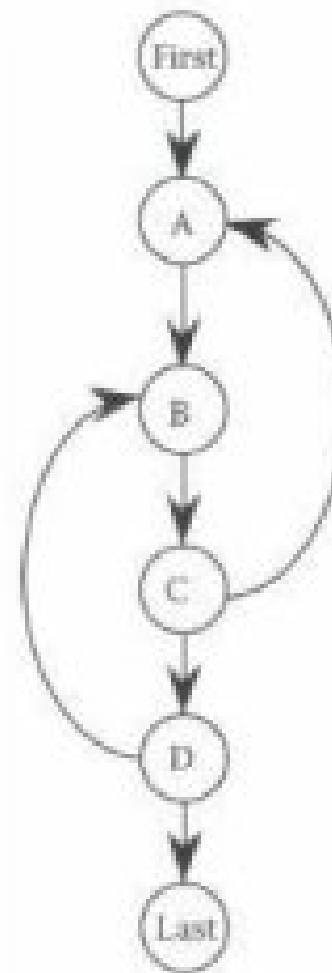


Fig.3: Concatenated loops



Nested loops



Knotted loops

- Every loop involves a decision, and we need to test both outcomes of the decision: one is to traverse the loop ,and the other is to exit the loop.
- Once the loop is tested ,the tester condenses it into single node.
- If loops are nested, this process is repeated starting with the innermost loop and working outward.
- If loops are knotted, it will be necessary to carefully analyze them in terms of the dataflow method.

Basis Path testing

- Basis Path Testing is a white-box testing technique that: Identifies a set of independent paths through a program and ensures that every possible statement, branch, and condition is executed at least once.

McCabe's Basis Path Testing:

The method devised by McCabe to carry out basis path testing has four steps.

These are:

1. Compute the program graph.
2. Calculate the cyclomatic complexity.
3. Select a basis set of paths.
4. Generate test cases for each of these paths

- Step 1: To begin, we need a program graph from which to construct a basis. Figure 4 is a directed graph which is the program graph (or the DD-Path graph) of some program. The program does have a single entry (A) and a single exit (G).

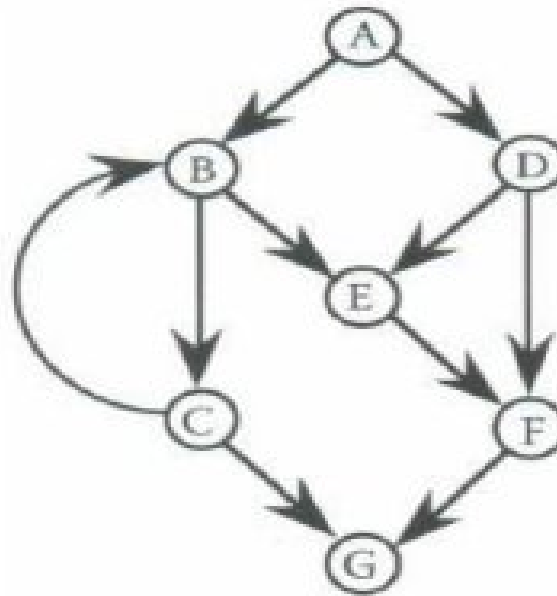
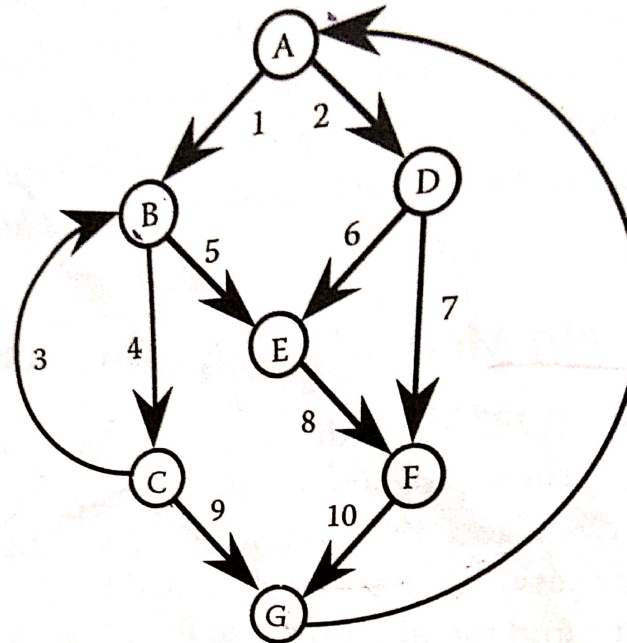


Fig. 4: McCabe's Control Graph

- We can create a strongly connected graph by adding an edge from the (every)sink node to the (every) source node. Figure 5 shows McCabe's derived strongly connected graph.



McCabe's derived strongly connected graph.

Step 2: In graph theory, the cyclomatic complexity is defined for a strongly connected graph is the number of linearly independent circuits in the graph. (A circuit is similar to a chain: no internal loops or decisions occur, but the initial node is the terminal node. A circuit is a set of 3-connected nodes.)

The formula for cyclomatic complexity is given by-

$$V(G) = e - n + p \quad \text{or} \quad V(G) = e - n + 2p$$

Where, **e**: is the number of edges

n: is the number of nodes

p: is the number of connected regions.

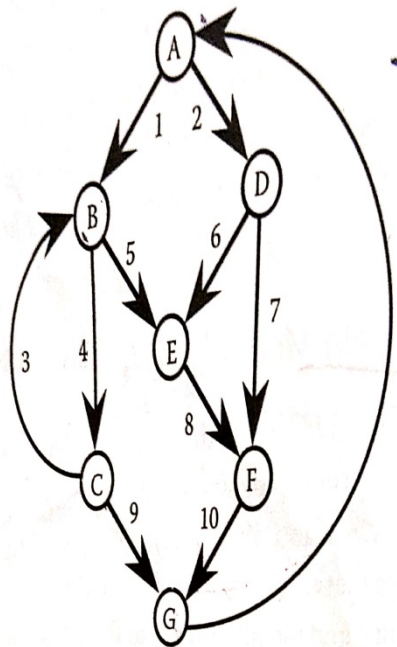
The number of **linearly independent paths** from the source node to the sink node in Fig. 4 is

$$V(G) = e - n + 2p = 10 - 7 + 2(1) = 5$$

The number of **linearly independent circuits** in the graph in Fig. 5 is

$$V(G) = e - n + p = 11 - 7 + 1 = 5$$

- The cyclomatic complexity of the strongly connected graph in Figure 5 is 5.
- Hence there are five linearly independent circuits.
- If the added edge is deleted from node G to node A, these five circuits become five linearly independent paths from node A to node G.
- Step3: An independent path is any path through the software that introduces at least one new set of processing statements or a new condition.
 - Independent Paths –
 - P1:A,B,C,G
 - P2 :A,B,C,B,C,G
 - P3: A,B,E,F,G
 - P4:A,D,E,F,G
 - P5:A,D,F,G



McCabe's derived strongly connected graph.

Table : Path / Edge Traversal

Path/Edges Traversed	1	2	3	4	5	6	7	8	9	10
p1: A, B, C, G	1	0	0	1	0	0	0	0	1	0
p2: A, B, C, B, C, G	1	0	1	2	0	0	0	0	1	0
p3: A, B, E, F, G	1	0	0	0	1	0	0	1	0	1
p4: A, D, E, F, G	0	1	0	0	0	1	0	1	0	1
p5: A, D, F, G	0	1	0	0	0	0	1	0	0	1
ex1: A, B, C, B, E, F, G	1	0	1	1	1	0	0	1	0	1
ex2: A, B, C, B, C, B, C, G	1	0	2	3	0	0	0	0	1	0

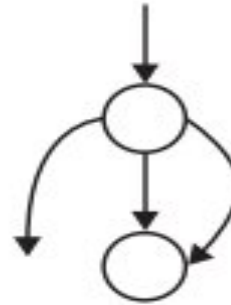
Structured programming constructs

- Condensation graphs are a way of simplifying an existing graph.
- The basic idea is to look for the graph of one of the structured programming constructs, collapse it into a single node, and repeat until no more structured programming constructs can be found.
- This process is followed in Figure 8.12, which starts with the DD-path graph of the pseudocode triangle program. The if–then–else construct involving nodes B, C, D, and E is condensed into node a, and then the three if–then constructs are condensed onto nodes b, c, and d. The remaining if–then–else (which corresponds to the IF IsATriangle statement) is condensed into node e, resulting in a condensed graph with cyclomatic complexity $V(G) = 1$.

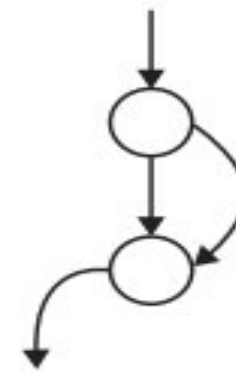
Sequence



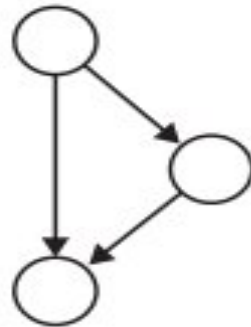
Pretest loop



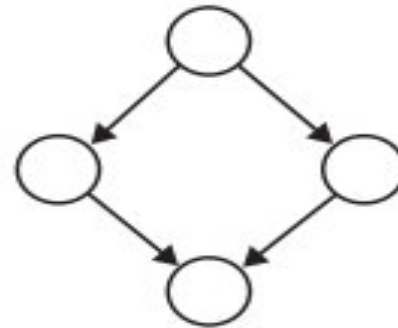
Posttest loop



If-Then



If-Then-Else



Cases/switch

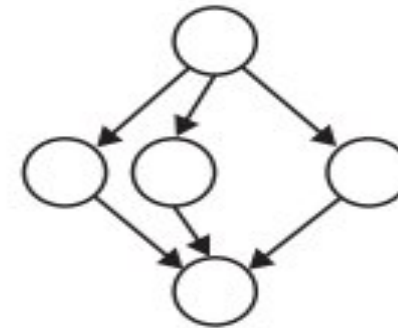


Figure 8.11 Structured programming constructs.

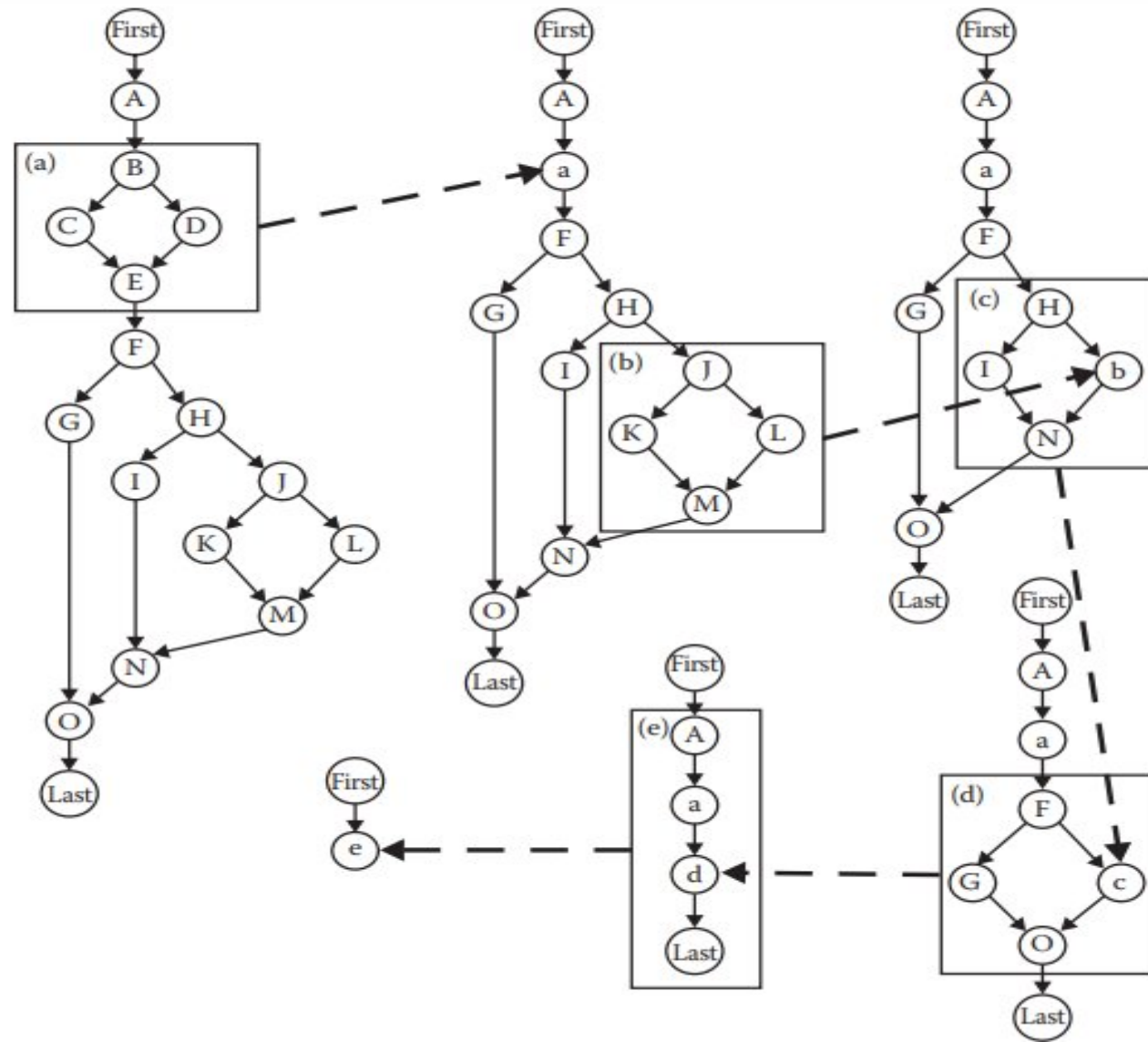


Figure 8.12 Condensing with respect to structured programming constructs.

Data Flow testing

- Dataflow testing refers to forms of structural testing that focus on the points at which variables receive values and the points at which these values are used.
- Most programs deliver functionality in terms of data.
- Variables that represent data somehow receive values and these values are used to compute values for other variables.
- Early dataflow analyzers often centered on a set of faults that are now known as define/reference anomalies:
 - A variable that is defined but never used(referenced).
 - A variable that is used before it is defined.
 - A variable that is defined twice before it is used.

Data Flow Testing – Basic Idea

- Data-flow testing involves selecting entry/exit paths with the objective of covering certain data definition and use patterns, commonly known as data-flow criteria
- An outline of data-flow testing is as follows:
 - Draw a data flow graph for the program
 - Select data-flow testing criteria
 - Identify paths in the data-flow graph to satisfy the selection criteria (i.e. all-defs, all-uses, all-P-uses/some-C-uses etc.)
 - Produce test cases for the selected paths

Define / Use Testing

The following definitions refer to a program P that has a program graph $G(P)$ and a set of program variables V . The program graph $G(P)$ is constructed with statement fragments as nodes and edges that represent node sequences. $G(P)$ has a single-entry node and a single-exit node. The set of all paths in P is $PATHS(P)$.

Definition: DEF (v, n)

Node $n \in G(P)$ is a defining node of the variable $v \in V$, written as $DEF(v, n)$, iff the value of the variable v is defined at the statement fragment corresponding to node n .

Ex: Input statements, assignment statements, loop control statements, and procedure calls. When the code corresponding to such statements executes, the contents of the memory location(s) associated with the variables are changed.

Definition: USE (v, n)

Node $n \in G(P)$ is a usage node of the variable $v \in V$, written as $USE(v, n)$, iff the value of the variable v is used at the statement fragment corresponding to node n .

Ex: Output statements, assignment statements, conditional statements, loop control statements and procedure calls. When the code corresponding to such statements executes, the contents of the memory location(s) associated with the variables remain unchanged.

Definition: (du-path)

A definition-use path with respect to a variable v (du-path) is a path in $PATHS(P)$ such that for some $v \in V$, there are define and usage nodes $DEF(v, m)$ and $USE(v, n)$ such that m and n are the initial and final nodes of the path.

Definition: (dc-path)

A definition-clear path with respect to a variable v (dc-path) is a definition-use path in $\text{PATHS}(P)$ with initial and final nodes $\text{DEF}(v, m)$ and $\text{USE}(v, n)$ such that no other node in the path is a defining node of v .

```

1  Program Commission (INPUT,OUTPUT)
2  Dim locks, stocks, barrels As Integer
3  Dim lockPrice, stockPrice, barrelPrice As Real
4  Dim totalLocks, totalStocks, totalBarrels As Integer
5  Dim lockSales, stockSales, barrelSales As Real
6  Dim sales, commission As Real
7  lockPrice = 45.0
8  stockPrice = 30.0
9  barrelPrice = 25.0
10 totalBarrels = 0
11 totalStocks = 0
12 totalBarrels = 0
13 Input(locks)
14 While NOT(locks = -1) "locks = -1 signals end of data
15   Input(stocks, barrels)
16   totalLocks = totalLocks + locks
17   totalStocks = totalStocks + stocks
18   totalBarrels = totalBarrels + barrels
19   Input(locks)
20 EndWhile
21 Output("Locks sold:," totalLocks)
22 Output("Stocks sold:," totalStocks)
23 Output("Barrels sold:," totalBarrels)
24 lockSales = lockPrice*totalLocks
25 stockSales = stockPrice*totalStocks
26 barrelsSales = barrelPrice * totalBarrels
27 sales = lockSales + stockSales + barrelSales
28 Output("Total sales: ", sales)
29 If (sales > 1800.0)
30   Then
31     commission = 0.10 * 1000.0
32     commission = commission + 0.15 * 800.0
33     commission = commission + 0.20*(sales-1800.0)
34   Else If (sales > 1000.0)
35     Then
36       commission = 0.10 * 1000.0
37       commission = commission + 0.15*(sales-1000.0)
38     Else
39       commission = 0.10 * sales
40     EndIf
41   EndIf
42 Output("Commission is $", commission)
43 End Commission

```

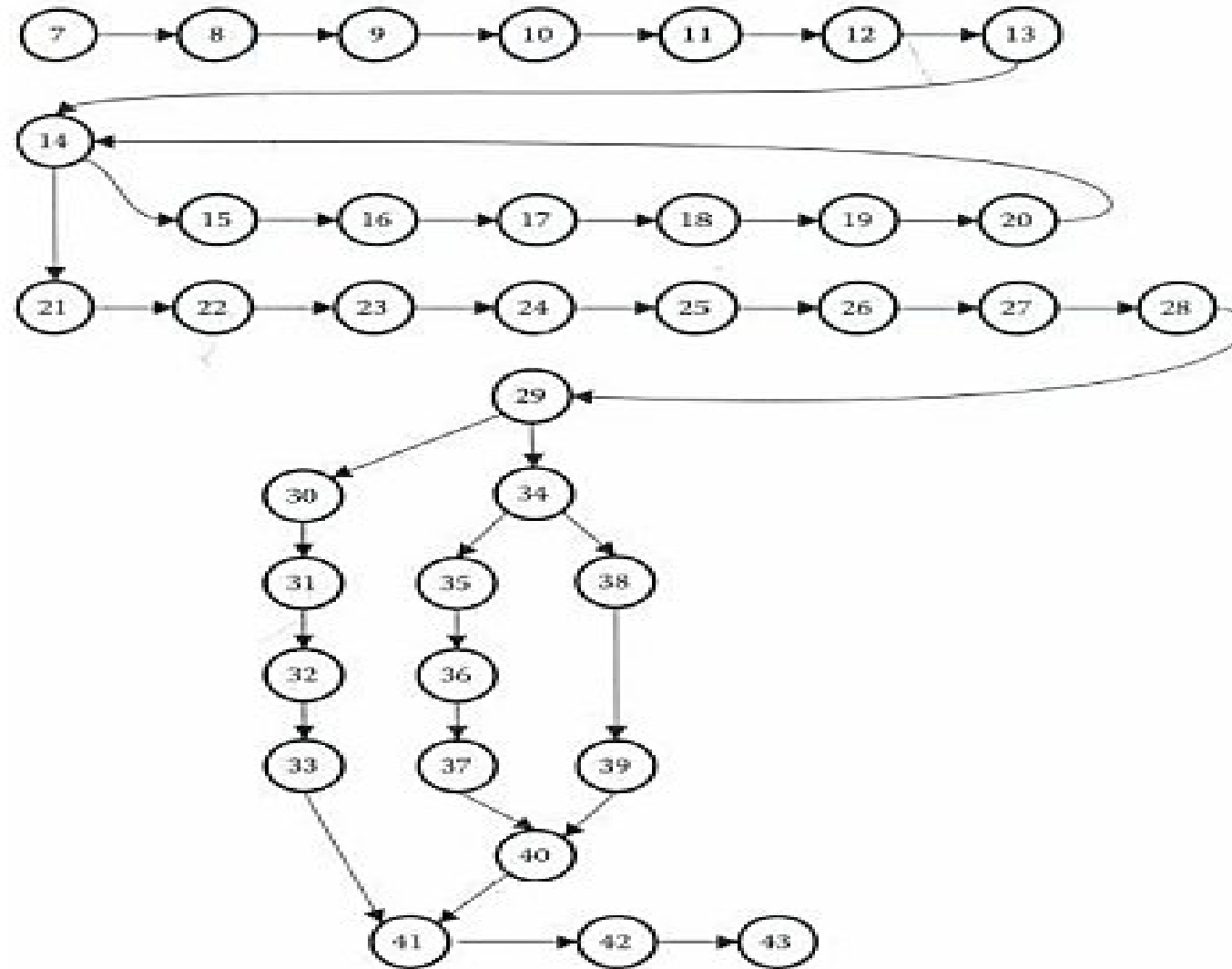


Fig. 9 Program graph of the commission program

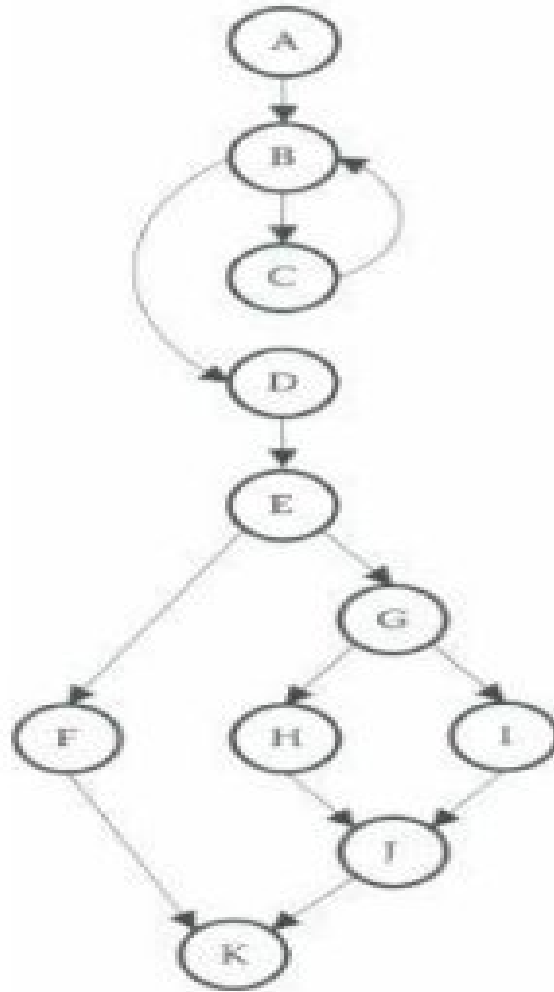


Fig. 10: DD-Path graph of the commission program

Table 9.1 DD-paths in Figure 9.1

<i>DD-path</i>	<i>Nodes</i>
A	7, 8, 9, 10, 11, 12, 13
B	14
C	15, 16, 17, 18, 19, 20
D	21, 22, 23, 24, 25, 26, 27, 28
E	29
F	30, 31, 32, 33
G	34
H	35, 36, 37
I	38
J	39
K	40
L	41, 42

Du-paths for Stocks

- First, let us look at a simple path: the du-path for the variable stocks. We have DEF(stocks, 15) and
- USE(stocks, 17), so the path $\langle 15, 17 \rangle$ is a du-path with respect to stocks. No other defining nodes are used for stocks; therefore, this path is also definition clear.

Du-paths for Locks

- Two defining and two usage nodes make the locks variable more interesting: we have DEF(locks, 13), DEF(locks, 19), USE(locks, 14), and USE(locks, 16). These yield four du-paths; they are
- shown in Figure 9.3.
- $p1 = \langle 13, 14 \rangle$
- $p2 = \langle 13, 14, 15, 16 \rangle$
- $p3 = \langle 19, 20, 14 \rangle$
- $p4 = \langle 19, 20, 14, 15, 16 \rangle$

These yield 6 du-paths:

$p5 = \langle 10, 11, 12, 13, 14, 15, 16 \rangle$ is a du-path in which the initial value of totalLocks (0) has a computation use. This path is definition-clear.

$p6 = \langle 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 14, 21 \rangle$ Path $p6$ ignores the possible repetition of the While loop (subpath $\langle 16, 17, 18, 19, 20, 14, 15 \rangle$) might be traversed several times. But $p6$ is a du-path that fails to be definition-clear.

$p7 = \langle 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 14, 21, 22, 23, 24 \rangle$

$p7 = \langle p6, 22, 23, 24 \rangle$ Du-path $p7$ is not definition-clear because it includes node 16. $p8 =$

$\langle 16, 17, 18, 19, 20, 14, 21 \rangle$

$p9 = \langle 16, 17, 18, 19, 20, 14, 21, 22, 23, 24 \rangle$ Both $p8$ & $p9$ are definition-clear

4. du-Paths for Sales

Only one defining node is used for sales therefore, all the du-paths with respect to sales must be definition-clear.

$p_{10} = \langle 27, 28 \rangle$

$p_{11} = \langle 27, 28, 29 \rangle$

$p_{12} = \langle 27, 28, 29, 30, 31, 32, 33 \rangle$

$p_{13} = \langle 27, 28, 29, 34 \rangle$

$p_{14} = \langle 27, 28, 29, 34, 35, 36, 37 \rangle$

$p_{15} = \langle 27, 28, 29, 34, 38, 39 \rangle$

Table Define/Use Paths for Commission

<i>Variable</i>	<i>Path (Beginning, End) Nodes</i>	<i>Feasible?</i>	<i>Definition-Clear?</i>
commission	31, 32	Yes	Yes
commission	31, 33	Yes	No
commission	31, 37	No	n/a
commission	31, 42	Yes	No
commission	32, 32	Yes	Yes
commission	32, 33	Yes	Yes
commission	32, 37	No	n/a
commission	32, 42	Yes	No
commission	33, 32	No	n/a
commission	33, 33	Yes	Yes
commission	33, 37	No	n/a
commission	33, 42	Yes	Yes
commission	36, 32	No	n/a
commission	36, 33	No	n/a
commission	36, 37	Yes	Yes
commission	36, 42	Yes	No
commission	37, 32	No	n/a
commission	37, 33	No	n/a
commission	37, 37	Yes	Yes
commission	37, 42	Yes	Yes
commission	38, 32	No	n/a
commission	38, 33	No	n/a
commission	38, 37	No	n/a
commission	38, 42	Yes	Yes

Slice-Based Testing

- A program slice is a set of program statements that contributes to or affects a value for a variable at some point in the program.
- A program P that has a program graph $G(P)$ and a set of program variables V . The nodes in $P(G)$ to refer to statement fragments.

Example: The commission problem

1. Slices on the locks: Slices on the locks variable show why it is potentially fault-prone. It has a P-use at node 14 and a C-use at node 16 and has two definitions, the I-defs at nodes 13 and 19.

S1: $S(\text{locks}, 13) = \{13\}$

S2: $S(\text{locks}, 14) = \{13, 14, 19, 20\}$

S3: $S(\text{locks}, 16) = \{13, 14, 19, 20\}$

S4: $S(\text{locks}, 19) = \{19\}$

2. Slices for stocks and barrels:

S5: $S(\text{stocks}, 15) = \{13, 14, 15, 19, 20\}$

S6: $S(\text{stocks}, 17) = \{13, 14, 15, 19, 20\}$

S7: $S(\text{barrels}, 15) = \{13, 14, 15, 19, 20\}$

S8: $S(\text{barrels}, 18) = \{13, 14, 15, 19, 20\}$

3. Slices for totalLocks:

S9: $S(\text{totalLocks}, 10) = \{10\}$

S10: $S(\text{totalLocks}, 16) = \{10, 13, 14, 16, 19, 20\}$

S11: $S(\text{totalLocks}, 21) = \{10, 13, 14, 16, 19, 20\}$

Slices S10 and S11 are equal because nodes 21 and 24 are an O-use and a C-use of totalLocks

4. Slices on totalStocks and totalBarrels:

S12: $S(\text{totalStocks}, 11) = \{11\}$

S13: $S(\text{totalStocks}, 17) = \{11, 13, 14, 15, 17, 19, 20\}$

S14: $S(\text{totalStocks}, 22) = \{11, 13, 14, 15, 17, 19, 20\}$

S15: $S(\text{totalBarrels}, 12) = \{12\}$

S16: $S(\text{totalBarrels}, 18) = \{12, 13, 14, 15, 18, 19, 20\}$

S17: $S(\text{totalBarrels}, 23) = \{12, 13, 14, 15, 18, 19, 20\}$

5. Assignment statements:

S18: $S(\text{lockPrice}, 24) = \{7\}$

S19: $S(\text{stockPrice}, 25) = \{8\}$

S20: $S(\text{barrelPrice}, 26) = \{9\}$

S21: $S(\text{lockSales}, 24) = \{7, 10, 13, 14, 16, 19, 20, 24\}$

S22: $S(\text{stockSales}, 25) = \{8, 11, 13, 14, 15, 17, 19, 20, 25\}$

S23: $S(\text{barrelSales}, 26) = \{9, 12, 13, 14, 15, 18, 19, 20, 26\}$

6. Slices on sales and commission:

Only one defining node exists for sales, the A-def at node 27. The remaining slices on sales show the P-uses, C-uses, and the O-use in definition-clear paths.

S24: $S(\text{sales}, 27) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S25: $S(\text{sales}, 28) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S26: $S(\text{sales}, 29) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S27: $S(\text{sales}, 33) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S28: $S(\text{sales}, 34) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S29: $S(\text{sales}, 37) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

S30: $S(\text{sales}, 39) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27\}$

Six A-def nodes are used for commission. Three computations of commission are controlled by P-uses of sales in the IF, ELSE IF logic. This yields three paths of slices that compute commission.

S31: $S(\text{commission}, 31) = \{31\}$

S32: $S(\text{commission}, 32) = \{31, 32\}$

S33: $S(\text{commission}, 33) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 29, 30, 31, 32, 33\}$

S34: $S(\text{commission}, 36) = \{36\}$

S35: $S(\text{commission}, 37) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 36, 37\}$

S36: $S(\text{commission}, 39) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 29, 34, 38, 39\}$

S37: $S(\text{commission}, 41) = \{7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39\}$

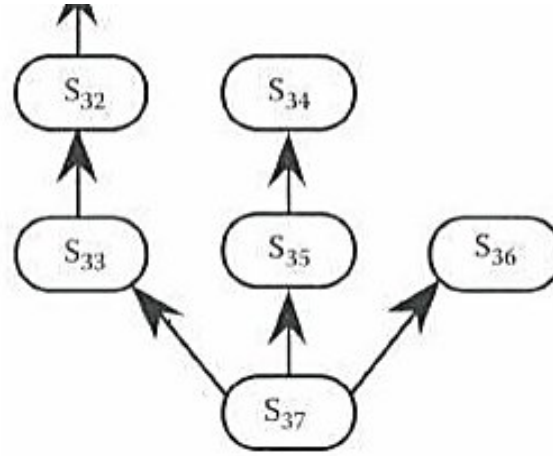


Fig. 11: lattice of slices on commission

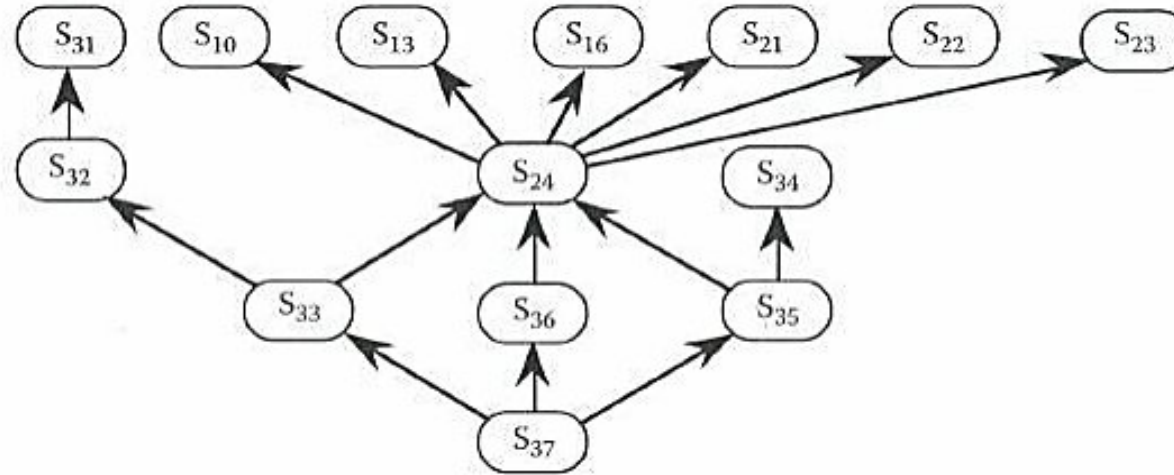


Fig. 12: lattice on sales and commission

Test Execution

Overview

- A product build is not complete until it has passed a suite of test cases
- For unit, integration and system testing, the software under test may be combined with additional “scaffolding” code to provide a suitable test environment
- Test scaffolding typically includes automated test oracles
- The test environment often includes additional support for selecting test cases and for summarizing and reporting results

Scaffolding

- Code developed to facilitate testing is called *scaffolding*.
- Scaffolding refers to the creation of temporary code or support structures that help test individual units or modules of a program.
- Scaffolding may include test drivers, test harnesses and stubs in addition to program instrumentation and support for recording and managing test execution

Types of Scaffolding Components:

Component	Purpose	Example	
Test Drivers	Substitute for the main program or calling module that invokes the component under test.	A small test program that calls a sorting function with test data.	
Stubs	Replace called functions or components not yet implemented.	A stub simulating a hardware device or a database call.	
Test Harnesses	Substitute for parts of the deployment environment (e.g., middleware, network layer, etc.).	A harness simulating the production server to test web APIs locally.	

- The purpose of scaffolding are to..
 - provide controllability to execute test cases and
 - Observability to judge the outcome of test execution
- Sometimes scaffolding is required to simply make a module executable
- It may be desirable to substitute a separate test “driver” program for the full system, in order to provide more direct control of an interface or to remove dependence on other subsystems

- Consider, for example, an interactive program that is normally driven through a GUI.
- Assume that each night the program goes through a fully automated and unattended cycle of integration, compilation, and test execution
- It is necessary to perform some testing through the interactive interface, but it is neither necessary nor efficient to execute all test cases that way

- Small driver programs, independent of the GUI, can drive each module through large test suites in a short time
- When testability is considered in software architectural design, it often happens that interfaces exposed for use in scaffolding have other uses.
- for example, the interfaces needed to drive an interactive program without its GUI are likely serve also as the interface for a scripting facility

Generic versus specific scaffolding

- The simplest form of scaffolding is a driver program that runs a single, specific test case
- Generic scaffolding may suffice for small numbers of hand-written test cases
- A large suite of automatically generated test cases and a smaller set of hand written test cases can share the same underlying generic test scaffolding

- Scaffolding to replace portions of the system is somewhat more demanding
- Again both generic and application specific approaches are possible
- The simplest kind of stub is *mock*
- It can be generated automatically by analysis of the source code
- A mock is limited to checking expected invocations and producing pre computed results that are part of the test case specification or were recorded in a prior execution

Test oracles

- **Test oracle is a Software that applies a pass/fail criterion to a program execution.**
- **Used to determine whether a software test has passed or failed by comparing the actual output with the expected output.**
- Checking real time response against latency requirements or dealing with voluminous output data in a machine readable rather than human readable form.
- Test oracle would classify every execution of a correct program as passing and would detect every program failure.

- A test oracle may apply a pass/fail criterion that reflects only part of the actual program specification or is an approximation, and therefore passes some program executions it ought to fail.
- **A test oracle may also give false alarms, failing an execution that it ought to pass**
- False alarms in test execution are highly undesirable because ..
 - Of the direct expense of manually checking them
 - And they make it likely that real failures will be overlooked

- **A comparison-based oracle is a type of test oracle that works by: Comparing the actual output of the software under test with predicted (expected) output, which is known in advance.**
- **If the actual output matches the expected output → ✓ Test passes**
- **If it does not match → ✗ Test fails**
- A test case with a *comparison based oracle* relies on predicted output that is either precomputed as part of the test case specification or can be derived in some way independent of the program under test.
- Support for comparison based test oracles is often included in a test harness program or testing framework

- Harness typically takes two inputs:
 1. The input to the program under test
 2. The predicted output
- Frameworks for writing test cases as program code likewise provide support for comparison based oracles
- The assertEquals method of JUnit, is a simple example of comparison based oracle support

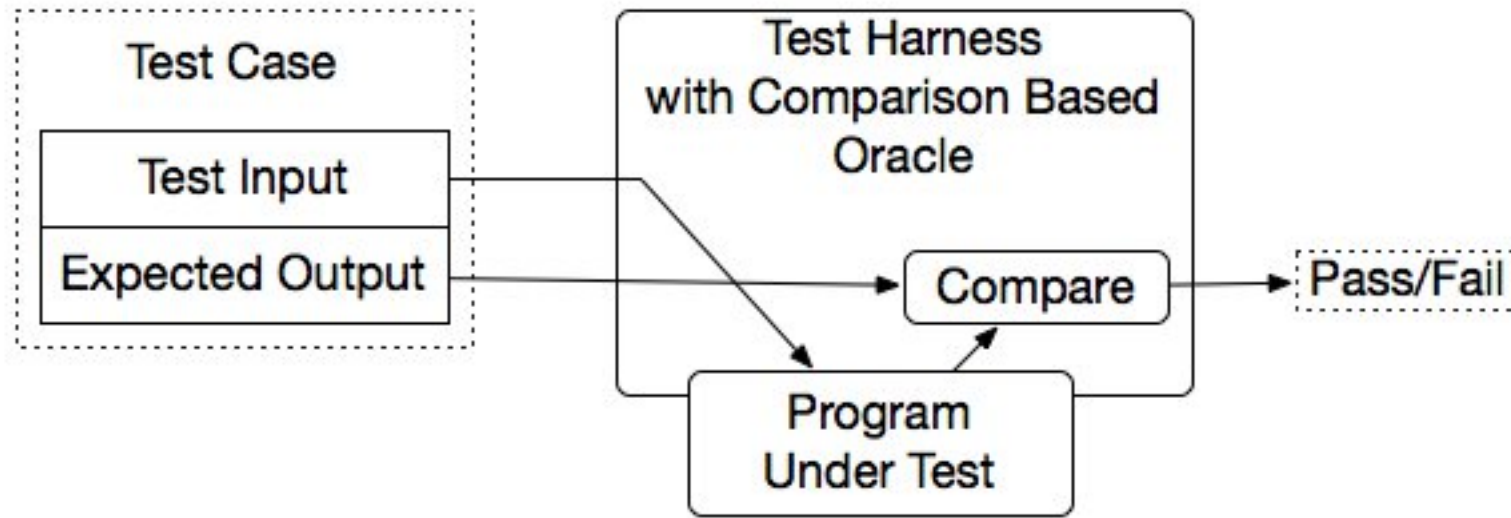
```

1  public final class IntCharSet {
75  ...
76  public void add(Interval interval) {
186  ...
187  }

1  package JFlex.tests;
2
3  import JFlex.IntCharSet;
4  import JFlex.Interval;
5  import junit.framework.TestCase;
11  ...
12  public class CharClassesTest extends TestCase {
25  ...
26  public void testAdd1() {
27      IntCharSet set = new IntCharSet(new Interval('a','h'));
28      set.add(new Interval('o','z'));
29      set.add(new Interval('A','Z'));
30      set.add(new Interval('h','o'));
31      assertEquals("{ ['A'-'Z'] ['a'-'z'] }", set.toString());
32  }
33
34  public void testAdd2() {
35      IntCharSet set = new IntCharSet(new Interval('a','h'));
36      set.add(new Interval('o','z'));
37      set.add(new Interval('A','Z'));
38      set.add(new Interval('i','n'));
39      assertEquals("{ ['A'-'Z'] ['a'-'z'] }", set.toString());
40  }
99  ...
100 }

```

Figure 17.1: Excerpt of JFlex 1.4.1 source code (a widely used open-source scanner generator) and accompanying JUnit test cases. JUnit is typical of basic test scaffolding libraries, providing support for test execution, logging, and simple result checking (as-
sertEquals in the example). The illustrated version of JUnit uses Java reflection to find and execute test case methods; later versions of JUnit use Java annotation (metadata) facilities, and other tools use source code preprocessors or generators.

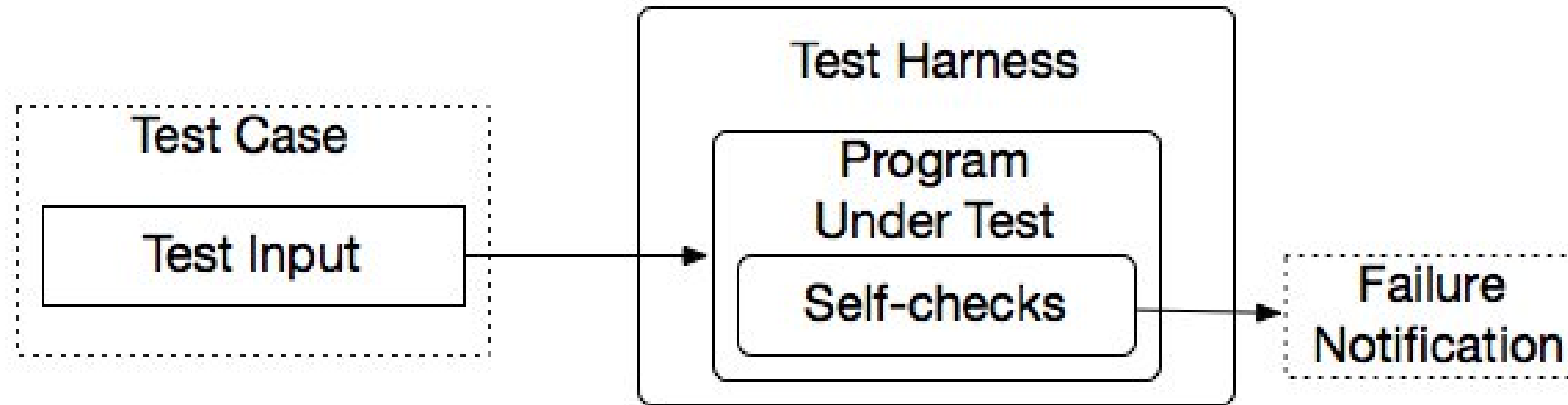


With a comparison-based oracle, we need predicted output for each input

Oracle compares actual to predicted output, and reports failure if they differ

Fine for a small number of hand-generated test cases

Self checks as oracles



- An oracle can also be written as *self-checks*. The program checks itself and acts as its own oracle.
- Instead of relying on an external oracle (like predefined expected outputs), the program has built-in checks — like assertions, error-handling— to determine if something has gone wrong.
- Advantages and limits: Usable with large, automatically generated test suites, but often only a *partial* check
 - e.g., structural invariants of data structures
 - recognize *many* or *most* failures, but not all

Capture and replay

- **Capture and Replay** is a testing technique where:
 1. **Capture:** You record the interactions, inputs, or events that happen during a real use of the software (e.g., user actions, network messages, system calls).
 2. **Replay:** You **automatically replay those recorded actions** later to test if the system behaves the same way or detect bugs.

Capture and replay

- Sometimes there is no alternative to human input and observation
 - Even if we separate testing program functionality from GUI, some testing of the GUI is required
- We can at least cut *repetition* of human testing
- *Capture* a manually run test case, *replay* it automatically
 - with a comparison-based test oracle: behavior same as previously accepted behavior
 - reusable only until a program change invalidates it
 - lifetime depends on abstraction level of input and output